BINARY TREE & BST

Complete Problem-Solving Roadmap

A systematic DSA reference for recognizing tree patterns, choosing traversals, designing recursion, and solving common Binary Tree and Binary Search Tree problems.

FOUNDATIONS	Tree terminology, classification, complexity
CORE PATTERNS	DFS, BFS, recursion, paths, validation, BST
ADVANCED TOPICS	Views, LCA, distance K, construction, serialization
SOLVING METHOD	A repeatable workflow, edge cases, and practice order

How to use these notes: First identify the problem pattern, then define what the helper function means, establish its base case, and only then write the recurrence or traversal.

A systematic reference for recognizing tree-problem patterns, choosing the right traversal, designing recursion, and implementing common binary-tree and BST algorithms.

---

1. Understand the Tree Before Coding

1.1 Basic terminology

1. **Root:** The topmost node.
1. **Parent / Child:** A node directly connected above / below another node.
1. **Leaf:** A node with no children.
1. **Subtree:** A node together with all of its descendants.
1. **Depth:** Number of edges from the root to a node.
1. **Height:** Number of edges on the longest downward path from a node to a leaf. Some problems define height in nodes instead; check the statement.
1. **Level:** Often depth + 1 when the root is level 1. Follow the problem's convention.
1. **Ancestor / Descendant:** A node above / below another node on a path.
1. **Diameter:** Longest path between any two nodes. The path may or may not pass through the root.
1. **Balanced tree:** Usually means the heights of the left and right subtrees of every node differ by at most one.

1.2 Common node definition

1.3 First questions to ask

Before writing code, identify:

1. Is the input a general binary tree or a Binary Search Tree (BST)?
2. Is the answer about every node, one target node, a root-to-leaf path, or a whole-tree property?
3. Does the problem require a traversal order?
4. Do I need information from children to calculate a parent result?
5. Do I need information from the root/ancestors to calculate a child result?
6. Is the answer a value, boolean, list, path, or modified tree?
7. Can the tree be empty? Can it contain one node, duplicate values, or negative values?
8. Does the problem define height/path length in edges or nodes?

---

2. Classify the Problem

Use the wording of the question to identify its likely pattern.

Problem wording / requirement	Likely pattern
Visit or return nodes in a particular order	Traversal
Process nodes level by level	BFS / Queue
Reverse, reflect, or compare two sides	Mirror / Symmetry
Find a value or target node	Search
Check whether a tree obeys a rule	Validation
Root-to-leaf, any-to-any, maximum/minimum sum	Path recursion
Build a tree from traversals or descriptions	Construction
Find common ancestor of two nodes	LCA
Find nodes at distance K / burn tree	Parent mapping + BFS
Top, bottom, left, right, vertical view	BFS + coordinates / level logic
Check whether one tree occurs inside another	Subtree comparison
Convert tree to string or restore it	Serialization / Deserialization
Modify links or flatten a tree	Pointer manipulation
Find kth smallest / predecessor / successor	BST inorder properties

2.1 The six broad categories

1. **Traversal:** Preorder, inorder, postorder, level order, zigzag, boundary, views.
2. **Mirror / Symmetric:** Invert a tree, check symmetry, compare mirrored subtrees.
3. **Search:** Find a node, path to a node, BST search, predecessor/successor.
4. **Validation:** Same tree, balanced tree, valid BST, complete tree, symmetry.
5. **Path Sum / Path Problems:** Root-to-leaf, any-to-any, maximum path, count paths.
6. **Tree Construction:** Build from traversals, sorted arrays, descriptions, or serialized data.

---

3. The Core Traversal Toolkit

3.1 Depth-First Search (DFS)

DFS explores a branch deeply before moving to another branch. Its three standard orders are:

Traversal	Order	Memory trick
Preorder	Node → Left → Right (NLR)	Node first
Inorder	Left → Node → Right (LNR)	Node in middle
Postorder	Left → Right → Node (LRN)	Node last

For a BST, inorder traversal returns values in nondecreasing order.

3.1.1 Preorder — Recursive

3.1.2 Preorder — Iterative, one stack

Push right before left because a stack is LIFO.

3.1.3 Inorder — Recursive

3.1.4 Inorder — Iterative, one stack

Keep moving left while saving ancestors. When there is no left child, process the top node and move right.

3.1.5 Postorder — Recursive

3.1.6 Postorder — Iterative, two stacks

The first stack creates Node → Right → Left processing in reverse collection order; the second stack reverses it to Left → Right → Node.

3.1.7 Postorder — Iterative, one stack

`lastVisited` tells us whether the right subtree has already been processed.

Important correction: Postorder is LRN. It is the reverse of NRL (Node → Right → Left), not the reverse of ordinary preorder NLR.

3.2 Breadth-First Search (BFS) / Level Order

Use a queue when the problem says level, minimum number of edges, nearest, or level-by-level.

Standard level order template

The key step is saving `q.size()` before processing a level. This prevents newly added children from being processed as part of the current level.

Common BFS variations

1. **Level averages:** Calculate sum and count for each level.
1. **Maximum per level:** Track the maximum while processing each level.
1. **Zigzag:** Alternate the direction in which each level is stored.
1. **Right / left view:** Record the last / first node of each level.
1. **Minimum depth:** Stop at the first leaf encountered.
1. **Level order from bottom:** Reverse the list of levels or insert levels at the front.
1. **Nodes at distance K:** Use a queue and visit parent, left, and right neighbours.

---

4. How to Design Recursive Solutions

Most binary-tree DFS problems use one of three recursion styles.

4.1 Pattern A — Return information from children

Use this when a node's answer depends on its left and right subtrees.

Flow:

1. Handle `root == null`.
2. Recursively solve the left subtree.
3. Recursively solve the right subtree.
4. Combine the returned results.
5. Return the answer for the current node.

Examples: height, size, balanced tree, diameter, maximum path sum, subtree properties.

4.2 Pattern B — Carry information from parent to child

Use this when a child needs information accumulated along the path from the root.

Examples: root-to-node path, root-to-leaf sum, path with constraints, ancestor information.

Java primitive values such as `int` are passed by value, so each recursive call receives its own copy of `sum`.

4.3 Pattern C — Global answer + local return value

Use this when the final answer may be located anywhere in the tree, but each node returns a smaller contribution to its parent.

Examples: diameter, maximum path sum, maximum path length under a condition.

Critical distinction: A path passing through the current node may use both children. A path returned to the parent can use only one child, because a simple path cannot branch.

4.4 Recursion checklist

For every recursive solution, answer these questions:

1. What does the function return?
2. What is the base case?
3. What should the left recursive call calculate?
4. What should the right recursive call calculate?
5. How do I combine both results?
6. Is the answer returned to the parent, or updated in a global variable?
7. Can the answer be negative or zero? Does the problem allow an empty path?
8. Am I counting nodes or edges?

---

5. Mirror, Symmetry, and Tree Comparison

5.1 Invert / Mirror a Binary Tree

Swap the left and right children at every node.

5.2 Check whether a tree is symmetric

Compare the left subtree with the right subtree in mirrored order.

5.3 Check whether two trees are identical

5.4 Check whether one tree is a subtree of another

At each node in the main tree, test whether the candidate subtree matches exactly.

---

6. Search and Root-to-Node Paths

6.1 Search in a normal binary tree

There is no ordering rule, so in the worst case every node must be checked.

6.2 Find a root-to-target path

Backtracking is useful when the answer must contain the nodes on the path.

The removal step is essential: it undoes the current choice when that branch does not contain the target.

---

7. Binary Search Tree (BST) Patterns

A BST follows an ordering rule. In the usual distinct-key definition, every value in the left subtree is smaller than the node, and every value in the right subtree is larger. Always check how the problem handles duplicates.

7.1 Search in a BST

7.2 Validate a BST using bounds

Checking only `root.left.val < root.val < root.right.val` is not enough. Every descendant must obey the ancestor's bounds too.

Adjust the comparisons if the problem explicitly permits duplicate values.

7.3 Kth smallest element

Inorder traversal of a BST visits values in sorted order.

7.4 Lowest Common Ancestor in a BST

Use the BST ordering to move toward the split point.

7.5 BST predecessor and successor

1. **Predecessor:** Greatest value smaller than the target.
1. **Successor:** Smallest value greater than the target.
1. If a left subtree exists, predecessor is the rightmost node in it.
1. If a right subtree exists, successor is the leftmost node in it.
1. Otherwise, use the ancestors while searching for the target.

7.6 Insert into a BST

This version ignores duplicate values. Follow the duplicate policy in the question.

7.7 Delete from a BST — cases to remember

1. **No child:** Remove the node.
2. **One child:** Replace the node with its only child.
3. **Two children:** Replace its value with the inorder successor (minimum in right subtree) or predecessor (maximum in left subtree), then delete that replacement node from its original location.

---

8. Validation and Tree Properties

8.1 Maximum depth / Height

8.2 Minimum depth

The minimum depth ends at a leaf. If one child is null, do not take the minimum with zero, because that would incorrectly treat the missing child as a leaf.

8.3 Count nodes

8.4 Check whether a tree is balanced

Return `-1` as soon as an unbalanced subtree is found; otherwise return its height.

8.5 Diameter of a binary tree

Diameter is the longest path between any two nodes. If measured in edges, the candidate through a node is `leftHeight + rightHeight`.

Reset the `diameter` variable for every new test case / invocation. If the problem asks for number of nodes rather than edges, convert the result according to its definition.

8.6 Check whether a tree is complete

A complete binary tree has every level completely filled except possibly the last, and the last level is filled from left to right.

BFS idea:

1. Push the root into a queue.
2. When a null child is seen, set a `gap` flag.
3. If a non-null node appears after `gap` becomes true, the tree is not complete.
4. Otherwise continue.

---

9. Path Sum and Other Path Problems

Always identify what kind of path the question means.

Path type	Meaning
Root-to-leaf	Starts at root and ends at a leaf
Root-to-node	Starts at root and ends at a specified node
Any-to-any	Can start and end at any nodes
Downward path	Moves from a node toward descendants
Maximum path	Usually any-to-any, with no repeated nodes

9.1 Root-to-leaf path sum — existence

9.2 Collect all root-to-leaf paths

Copy the path when storing it. Otherwise, later backtracking changes the stored result.

9.3 Maximum path sum — any-to-any

A path can use both branches at its highest point, but the value returned to the parent can use only one branch.

Use `Integer.MIN_VALUE` for the global answer so that an all-negative tree is handled correctly. Do not force the final answer to include zero unless the problem allows an empty path.

9.4 Count downward paths with a target sum

For problems such as counting all downward paths whose sum equals a target, use prefix sums and a frequency map.

Core idea:

1. Maintain the prefix sum from the root to the current node.
1. A previous prefix sum equal to `currentSum - target` identifies a valid path ending at the current node.
1. Add the current prefix sum before exploring children.
1. Remove it when backtracking so sibling branches do not affect one another.

Use `long` for sums when values may overflow `int`.

---

10. Lowest Common Ancestor (LCA) in a General Binary Tree

Unlike a BST, a normal binary tree has no ordering rule. Use recursive search.

This standard form assumes both target nodes exist in the tree. If the problem does not guarantee that, track whether both nodes were found before returning an LCA.

---

11. Tree Views and Coordinate-Based Problems

11.1 Left view and right view

Use level order traversal.

1. Left view: first node at every level.
1. Right view: last node at every level.

11.2 Top view and bottom view

Assign a horizontal distance (HD):

1. Root HD = 0.
1. Left child HD = parent HD - 1.
1. Right child HD = parent HD + 1.

Use BFS with a map from HD to node value.

1. **Top view:** Keep the first node encountered at each HD.
1. **Bottom view:** Replace the value as deeper nodes are encountered. Tie-breaking rules may vary by problem.

11.3 Vertical order traversal

Track both horizontal distance and depth/row.

1. Left child: `(HD - 1, row + 1)`.
1. Right child: `(HD + 1, row + 1)`.

Group nodes by HD. If the problem requires sorting nodes at the same position, apply its exact ordering rule (often row first, then value).

11.4 Boundary traversal

A common anti-clockwise boundary order is:

1. Root (unless it is the only node).
2. Left boundary excluding leaves.
3. All leaves from left to right.
4. Right boundary excluding leaves, in reverse order.

Avoid adding leaf nodes twice.

---

12. Parent Pointers, Distance K, and Burning Tree

In a normal binary tree, nodes do not store parent pointers. To move upward, first build a parent map.

General approach

1. Traverse the tree and map each node to its parent.
2. Locate the target node.
3. Start BFS from the target.
4. For every node, consider three neighbours: left child, right child, and parent.
5. Use a visited set to avoid revisiting nodes.
6. Stop after K BFS layers for distance K, or continue until the queue is empty for burning-time problems.

This transforms the tree into an undirected graph for BFS.

Important: Prefer node references as map/set keys when values can repeat.

---

13. Constructing a Binary Tree

13.1 Preorder + Inorder

1. Preorder's first element is the root.
1. Find that root in inorder.
1. Elements to its left form the left subtree; elements to its right form the right subtree.
1. Recursively build both subtrees.

Use a map from value to inorder index to avoid repeatedly scanning the inorder array. This standard method assumes unique values.

13.2 Inorder + Postorder

1. Postorder's last element is the root.
1. Find it in inorder.
1. Build the right subtree before the left subtree when consuming postorder backwards.

13.3 Preorder + Postorder

In general, preorder and postorder alone do not uniquely determine a binary tree. Additional information (for example, the tree is full) may make construction possible.

13.4 Sorted array to height-balanced BST

Choose the middle element as root, then recursively build the left and right halves.

13.5 Build tree from parent-child descriptions

Typical approach:

1. Create or reuse a node for every value.
2. Attach each child to the indicated left or right pointer.
3. Track all child values.
4. The root is the node that never appears as a child.

Check that the input does not describe conflicting parents or invalid relationships if validation is required.

---

14. Serialization and Deserialization

Serialization converts a tree into a representation that can be stored or transmitted. Deserialization reconstructs the same structure.

A common approach is preorder traversal with explicit null markers.

Example representation:

```
1,2,#,#,3,#,#
```

Here # represents a null child. Null markers are essential because values alone do not preserve the tree's shape.

For robust parsing:

1. Use a delimiter that cannot be confused with values, or escape values.
1. Preserve null markers.
1. Ensure deserialization consumes tokens in the same order used by serialization.

---

15. Flattening and Pointer-Manipulation Problems

Flatten binary tree to linked list

A common target order is preorder: Node → Left → Right.

One reverse-preorder strategy processes:

1. Right subtree.
2. Left subtree.
3. Current node.

Maintain a `prev` pointer:

1. Set `root.right = prev`.
1. Set `root.left = null`.
1. Update `prev = root`.

Another approach uses the rightmost node of the left subtree to connect the original right subtree, then moves the left subtree to the right. Choose the approach that matches the required complexity and constraints.

For pointer problems, draw the links before changing them. Save a reference before overwriting a pointer if that reference is still needed.

---

16. Common Mistakes and Edge Cases

16.1 Mistakes to avoid

1. Forgetting the null base case.
1. Assuming a binary tree is a BST.
1. Checking only immediate children when validating a BST.
1. Using `min(left, right)` for minimum depth when one child is missing.
1. Returning a two-branch path to a parent in diameter / maximum-path problems.
1. Forgetting to copy a backtracking path before storing it.
1. Forgetting to undo state when returning from recursion.
1. Processing newly added queue nodes in the same level by not capturing the queue size.
1. Confusing preorder NLR with the NRL order used to derive iterative postorder.
1. Using node values as unique identifiers when duplicates are possible.
1. Forgetting to reset global or instance variables between test cases.
1. Using `int` when path sums or accumulated values may overflow.
1. Assuming both LCA targets exist when the problem does not guarantee it.

16.2 Always test these cases

1. Empty tree.
2. Single-node tree.
3. Only-left-child chain.
4. Only-right-child chain.
5. Perfect / complete tree.
6. Unbalanced tree.
7. Duplicate values (if permitted).
8. Negative values.
9. All-negative path-sum tree.
10. Target missing from the tree.
11. Target equal to the root.
12. Very deep tree (consider recursion stack limits).

---

17. Complexity Cheat Sheet

Let:

1. `n` = number of nodes.
1. `h` = tree height.
1. `w` = maximum width of the tree.

Operation / pattern	Time	Extra space (typical)
DFS traversal	O(n)	O(h) recursive stack
Iterative DFS	O(n)	O(h) stack, up to O(n)
Level order BFS	O(n)	O(w) queue
Search in normal binary tree	O(n)	O(h) recursive stack
Search in balanced BST	O(log n)	O(log n) recursive stack
Search in skewed BST	O(n)	O(n) recursive stack
Height / count / balance	O(n)	O(h)
Diameter (optimized DFS)	O(n)	O(h)
Same tree / symmetry	O(n)	O(h)
Build using traversal arrays + index map	O(n)	O(n)
BST inorder kth smallest	O(h + k)	O(h)
Parent map + distance-K BFS	O(n)	O(n)

A skewed tree has height `O(n)`. A balanced tree has height `O(log n)`.

---

18. A Repeatable Workflow for Solving Any Tree Question

Follow this sequence in interviews and while practising.

Step 1 — Translate the requirement

Write one sentence describing exactly what must be returned.

Examples: - “Return values level by level.” - “Return whether every node satisfies the BST ordering rule.” - “Find the maximum sum of a simple path anywhere in the tree.”

Step 2 — Identify the pattern

Choose one: - DFS traversal - BFS / level order - Compare two trees - Search / path with backtracking - Return information from children - Carry information from ancestors - Global answer + local return - BST ordering - Parent map + BFS - Construction / serialization - Pointer manipulation

Step 3 — Define the helper contract

Write down what the recursive helper returns or what state it carries.

Examples: - `height(node)` returns the height of the subtree. - `isValid(node, low, high)` checks all values against inherited bounds. - `dfs(node, sum)` carries the sum from the root to the current node. - `gain(node)` returns the best one-branch contribution to the parent while updating a global maximum.

Step 4 — Write the base case

Usually: - `root == null` returns `0`, `false`, `true`, `null`, or an empty result depending on the problem. - Do not choose a base-case value mechanically. Choose the value that makes the recurrence correct.

Step 5 — Decide where the answer is formed

Ask: - Is the answer the result returned by the root? - Is it updated at every node? - Is it collected in a list? - Is it discovered by BFS? - Does it require backtracking?

Step 6 — Dry-run a small tree

Use a three-node tree and trace: - Function entry. - Left recursive call. - Right recursive call. - Current-node calculation. - Return value.

For BFS, write the queue after every poll and insertion.

Step 7 — Test edge cases

At minimum test an empty tree, one node, a skewed tree, and negative values when sums are involved.

Step 8 — State complexity

Explain why every node is visited once (or why the algorithm revisits nodes), and identify the maximum stack / queue / map size.

---

19. Quick Decision Guide

If the question asks…	Start with…
“Print / return nodes in order”	DFS traversal
“Each level / minimum steps”	BFS queue
“Mirror / symmetric”	Two-node recursive comparison
“Same tree / subtree”	Structural comparison
“Height / balanced / diameter”	Postorder DFS
“Maximum path sum”	Global answer + one-branch return
“Root-to-leaf sum / paths”	DFS with carried state and possibly backtracking
“Path to target”	DFS + backtracking
“Kth smallest in BST”	Inorder
“Valid BST”	Min/max bounds
“LCA in BST”	BST ordering
“LCA in normal tree”	Recursive left/right search
“Distance K / burning”	Parent map + BFS
“Top / bottom / vertical view”	BFS + horizontal distance
“Build from preorder and inorder”	Root selection + inorder index map
“Flatten / invert”	Pointer manipulation
“Encode / decode tree”	Serialization with null markers

20. Practice Order

Build the concepts in this order. Do not jump directly to advanced path problems before understanding the traversal and recursion foundations.

1. Preorder, inorder, postorder — recursive.
2. Preorder, inorder, postorder — iterative.
3. Level order, level averages, zigzag.
4. Maximum depth, minimum depth, count nodes.
5. Same tree, symmetric tree, invert tree, subtree.
6. Search and root-to-node path.
7. Balanced tree and diameter.
8. Root-to-leaf path sum and all root-to-leaf paths.
9. Maximum path sum and downward path-sum counting.
10. Left/right/top/bottom/vertical views and boundary traversal.
11. LCA in a binary tree and BST.
12. Distance K and burning tree.
13. Validate, search, insert, delete, kth smallest in BST.
14. Construct from traversals and sorted arrays.
15. Serialize / deserialize and flatten a tree.

Final Memory Rules

1. **DFS:** Think recursion or stack.
1. **BFS:** Think queue and fixed level size.
1. **Postorder:** Children first; ideal for height and subtree information.
1. **Preorder:** Parent first; useful for carrying state and building a representation.
1. **Inorder:** Sorted order in a BST.
1. **Backtracking:** Make a choice, explore, undo the choice.
1. **Global answer:** Use when the best answer may be anywhere in the tree.
1. **BST:** Use ordering to eliminate one side.
1. **Distance between nodes:** Add parent links, then use BFS.
1. **Construction:** Identify which traversal reveals the root and which identifies subtree boundaries.

The most important habit: define exactly what your helper function means before writing its code. Once the helper contract, base case, and return value are clear, most tree problems become a controlled application of one of these patterns.